

3D Printing Sales Tracker

Noah Culley
IT280

Changelog

Version 1.0

- Implemented core sales entry functionality
- Added user input fields for item name, price, quantity, and date
- Implemented DataGridView to display sales data
- Added total revenue calculation feature
- Switched to BindingList for real-time UI updates
- Disabled manual row entry in DataGridView for cleaner interface

1. Introduction

1.1 Purpose

The purpose is to make an app that's easy for a user to log every sale they receive quickly on any device. Taking only a few clicks to add a sale, it then automatically adds it to the app's database with sale info and timestamps. Later, the user can go back to the web app and look over their sales on a spreadsheet, showing features like totals and popular picks for easy restocking. Overall, my idea will allow the user to add sale info to a spreadsheet quickly, and then be able to retrieve all that information in an organized list.

1.2 Scope

This application will allow users to input sales data such as item name, price, quantity, date, etc. The system will store this data and provide basic summaries such as total revenue and item performance. The app could also allow exporting data to a spreadsheet-compatible format.

1.3 Intended Audience

This project is intended for small business owners, specifically individuals running one all by themselves, like my 3D printing business.

2. Overall Description

2.1 Product Perspective

This is going to be a standalone application that replaces manual spreadsheet tracking with a more streamlined and user-friendly interface.

2.2 Product Functions

The system will:

- Allow users to add new sales entries
- Store and retrieve sales data
- Display all sales in a structured format
- Calculate total revenue
- Identify top-selling items
- Export data to CSV format
- The system now supports real-time display of sales data and automatic calculation of total revenue.

2.3 User Characteristics

Users are expected to have basic computer knowledge, but do not need technical expertise.

2.4 Constraints

- Built using C# and Windows Forms (or ASP.NET if expanded)
- Limited to local storage initially
- 100-hour total time requirement

3. Specific Requirements

3.1 Functional Requirements

The system shall allow users to input:

- Item name
- Price
- Quantity

- Date
- The system shall store sales data for later retrieval
- The system shall display all sales in a list or table
- The system shall calculate:
 - Total revenue
 - Total number of sales
- The system shall allow exporting data to a CSV file

3.2 Non-Functional Requirements

- The interface should be simple and easy to use
- The system should respond quickly to user input
- Data should persist between sessions (saved and loaded)

4. Difficult Areas / Risks

4.1 Data Storage and Persistence

One of the difficulties with the project will be ensuring that data is saved and loaded correctly. This includes choosing between file storage and/or a database, handling potential errors when reading or writing data.

4.2 Data Organization and Accuracy

Making sure that all sales data is correctly stored and calculated (such as totals and item counts)

4.3 Exporting to Spreadsheet Format

Converting stored data into a CSV format that works correctly with spreadsheet software may be difficult, particularly in formatting and ensuring compatibility. So I will be working on this last, to better focus on the functionality of the app itself first.

4.4 User Interface Design

Designing an interface that is both simple and efficient without becoming cluttered may require iteration and testing. So I would need to keep it clean while also avoiding overcrowding the screen for the user.

5. Future Enhancements

- Add charts and graphs for sales trends
- Implement category tracking for products
- Connect to an online database
- Convert into a web-based application
- Add user accounts and login functionality